

Lab 04:

HW 4 (Multicycle CPU)

Hunjun Lee

hunjunlee@hanyang.ac.kr



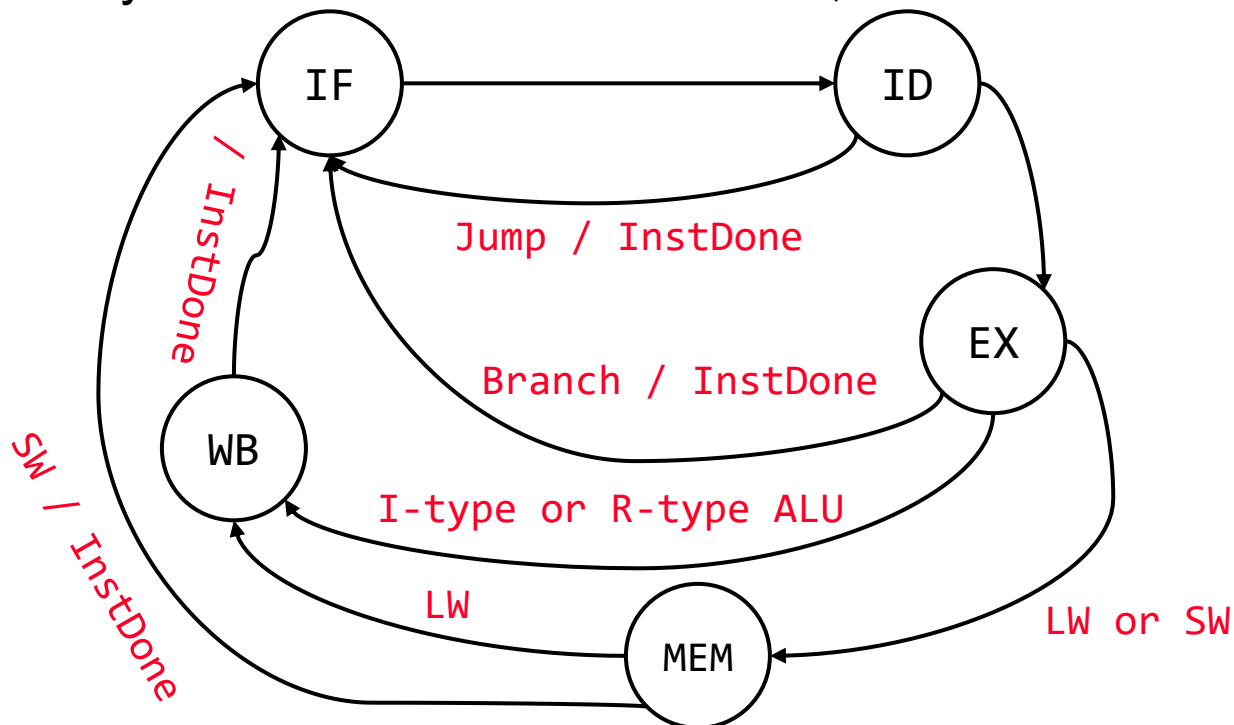
Overview

- ◆ You will upgrade your single cycle CPU into a multicycle CPU
- ◆ What to do?
 - Define additional control signals for multi-cycle architecture
 - Build an FSM to control multi-cycle execution
 - Rearchitect the datapath and finalize your design



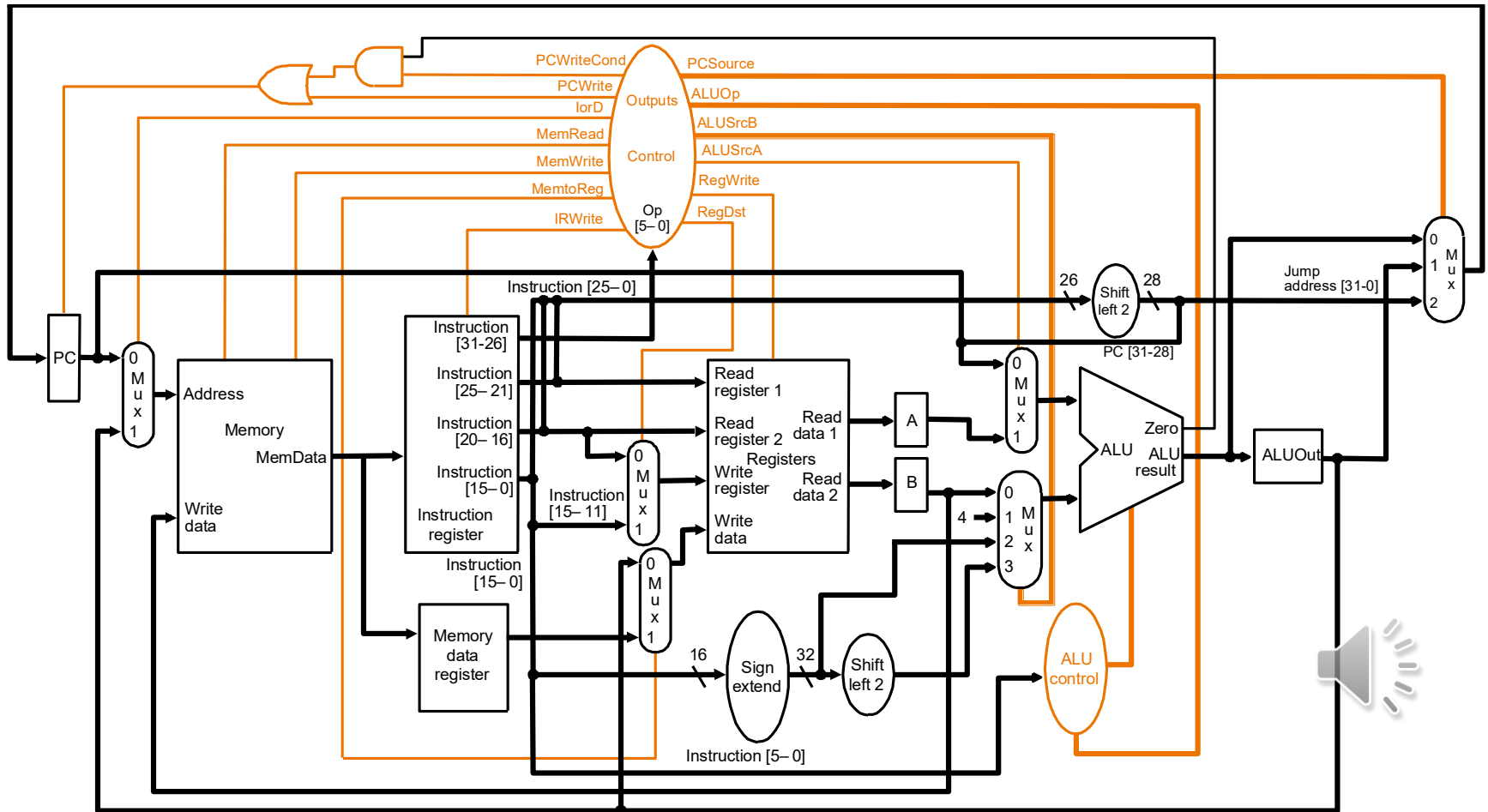
Multicycle Implementation

- ◆ You will be implementing a **up-to five-stage multicycle CPU**
- ◆ Each stage will take a single clock cycle
 - IF -> ID -> EX -> MEM -> WB
- ◆ Refer to the following FSM
 - Define your own FSM for JR and JAL, ...



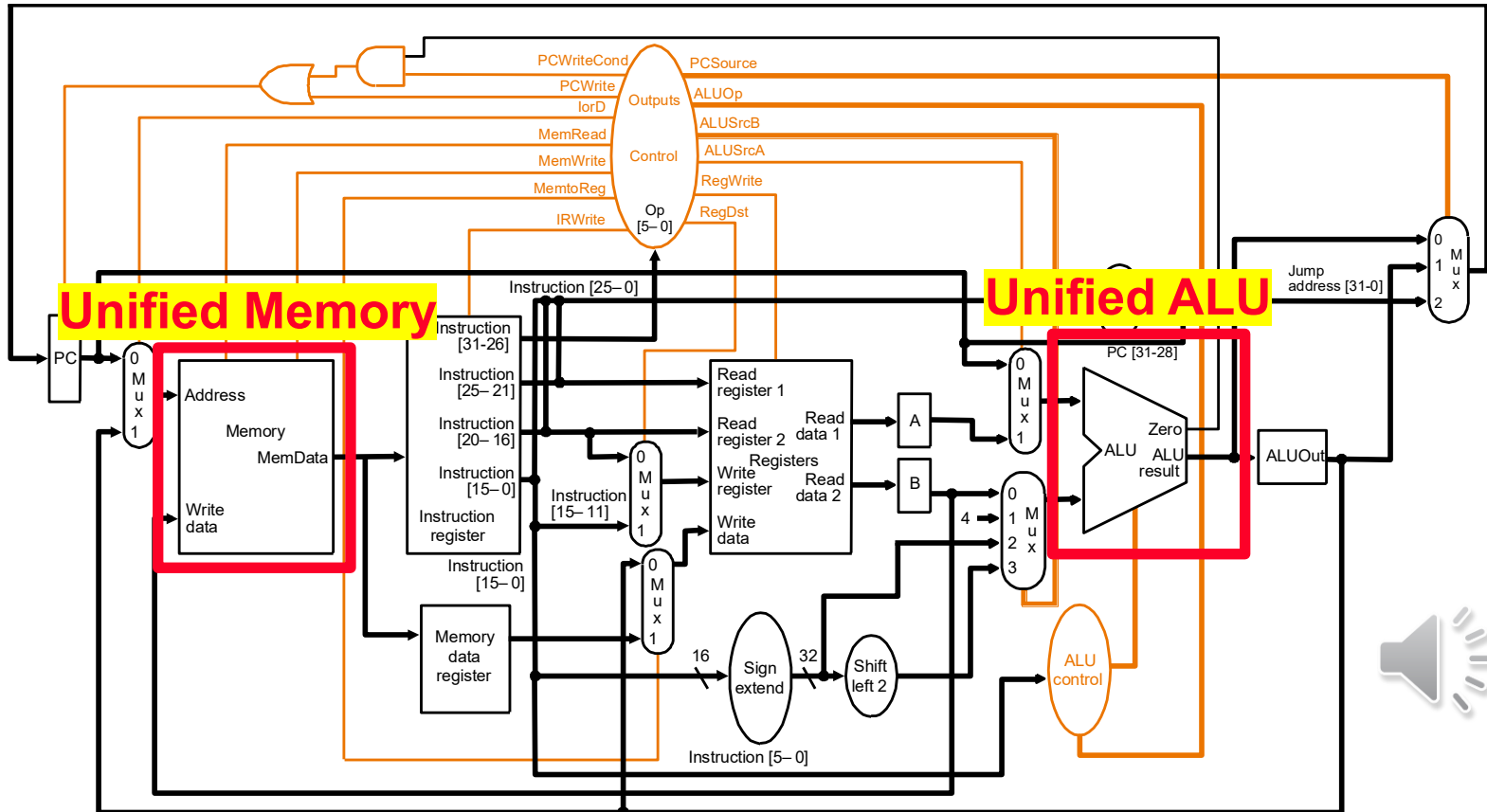
Target CPU Microarchitecture - 1

- ◆ Implement what you learned in the class, but again you need to **support jal and jr**



Target CPU Microarchitecture - 2

- ◆ Also, you should **share resources** among different computation stages (penalty if you do not or only partially share resources)



Assignment

◆ Files:

- GLOBAL.v (HW3)
 - Include constant values (opcode, funct ...)
 - You may change it to define states
- ALU.v (HW1)
- RF.v (HW2)
- MEM.v (Provided)
 - Modify the memory module (make it share a single port)
- CTRL.v (Modify HW3)
 - A controller to determine control signals
- CPU.v (Modify HW3)
 - The top module to combine these modules
- CPU_tb.v (HW3)
 - I gave you the test files for HW3 (Use it)



Unified Memory Module Implementation

- ◆ Now, your memory module should integrate
 - instruction addr w/ memory addr
 - read data w/ inst

```
`timescale 1ns / 1ps

module MEM(
    input          clk,
    input          rst,
    input [31:0]   mem_addr,
    input          memwrite,
    input [31:0]   mem_write_data,
    output reg [31:0] mem_read_data
);

    reg [31:0] memory [0:8191];

    initial begin
        $readmemh("initial_mem.mem", memory);
    end

    always @(*) begin
        mem_read_data = memory[(mem_addr >> 2)];
    end

    always @(posedge clk) begin
        if (!rst)
            if (MemWrite)
                memory[(mem_addr >> 2)] <= mem_write_data;
    end

endmodule
```



Submission

- ◆ Submission (Zip all the files)
 - For a two-people team: lab4_student_id1_student_id2.zip
 - For a one-person team: lab4_student_id.zip
 - You must follow the format (-10% for wrong file format)
 - Example:
 - lab4_2020102030.zip
 - lab4_2020102030_2022103040.zip
 - The zip file should contain:
 - every Verilog files except for the testbench
 - lab4_report.pdf



Submission

- ◆ You need to write a 4+-page report
 - Explain the overall structure and how you implemented each program
 - What's your FSM to implement JAL and JR instruction?
 - Draw the hardware modules if needed ...
 - How did you implement JAL / JR instruction and modified the uarchitecture?

- ◆ Due: Mon. May 11th
 - 1 week delay: -20%
 - Further delay: 0 point

